

統括管理者による登録ユーザーの権限変更機能

今回の要件における最大のポイントは、「自身（ログイン中の統括管理者）の権限変更はブロックする」という安全ガード（セーフティ）の設計です。

自身の権限をうっかり一般ユーザーに変えてしまうと、システムを管理できる人が誰もいなくなってしまう「詰み」の状態が起きてしまうため、実務システムでも絶対に外せない鉄則の防壁です。

この機能をスマートに実現するための**バックエンド（Java）** および**フロントエンド（Vue.js）**の修正・追加内容について提示します。

1. 権限変更のデータトラフィックと安全ガードの考察

権限変更のトラフィックは次のように組み立てます。

【フロントエンド】

1. ログインユーザーが「統括管理者」のときだけ、アカウント一覧テーブルに「権限変更用のセレクトボックス」を出現させる。
2. ただし、その行の `accountId` が自分自身の `ID` と一致する場合は、セレクトボックスを非活性 (`disabled`) にして操作不能にする。

【バックエンド】

3. フロントから「対象者のID」と「新しい権限名 (Role)」が届く。
4. 安全の二重防壁として、リクエストを出した本人 (ログインユーザー) のIDと対象者のIDが同じなら「却下 (400 Bad Request)」を返す。
5. 問題なければ、対象ユーザーの権限カラムを上書き保存する。

今回は、アカウント情報を司る既存の `UserAccount.java` (Entity) にすでに権限を管理するフィールド (例: `role` や `authority` など) が存在している想定で、コントローラーとサービスを構築します。

2. バックエンド（Java）側の修正コード

① Service層へのメソッド追加： `UserAccountService.java`

指定されたユーザーの権限を安全に上書きするメソッドを追加します。

```
/**
 * 統括管理者ロジック：指定されたユーザーの権限(Role)を更新する
 */
public void updateUserRole(String targetAccountId, String newRole) {
    UserAccount account = repository.findById(targetAccountId)
        .orElseThrow(() -> new RuntimeException("指定されたユーザーが見つかりません。ID: " + targetAccountId));

    // 権限フィールドに新しい値をセット（例: "ADMIN" => isAuth:1, "USER" => isAuth:0）
    int newIsAuth = newRole.equals("ADMIN") ? 1 : 0;
    account.setIsAuth(newIsAuth);

    repository.save(account);
    logger.info(
        "=== [Service] ユーザー権限を更新しました (ID: " + targetAccountId + ", isAuth: " + account.getIsAuth() + ") ===");
}
```

② Controller層へのエンドポイント追加： `UserAccountController.java`

リクエスト元のIDと変更対象のIDを比較するセーフティガードを敷いた受付窓口を新設します。

```
/**
 * 管理者用：統括管理者からのユーザー権限変更リクエストの受付
 * POST http://localhost:8080/api/users/admin/update-role
 */
@PostMapping("/admin/update-role")
public ResponseEntity<String> changeUserRole(@RequestBody Map<String, Object> payload) {
    String loginAccountId = (String) payload.get("loginAccountId"); // 操作している本人のID
    String targetAccountId = (String) payload.get("targetAccountId"); // 変更対象のユーザーID
    String newRole = (String) payload.get("newRole"); // 新しい権限文字列

    // バリデーション
    if (targetAccountId == null || newRole == null) {
        return ResponseEntity.badRequest().body("【却下】必要なパラメータが不足しています。");
    }

    // 【最重要安全ガード】変更対象が自分自身である場合は、バックエンド側でも絶対に拒否する
    if (targetAccountId.equals(loginAccountId)) {
        return ResponseEntity.badRequest().body("【システム保護】自分自身の管理権限を変更することはできません。");
    }

    try {
        userService.updateUserRole(targetAccountId, newRole);
        return ResponseEntity.ok("ユーザーの権限を正常に更新しました。");
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body("権限更新中にエラーが発生しました: " + e.getMessage());
    }
}
```

3. フロントエンド（Vue.js）側の修正コード

既存の「アカウント一覧・退会管理」タブを表示する `AdminUserList.vue` を修正します。
変更箇所が広範囲に及ぶ為、今回はソースコードのうち `<template>` と `<script>` の全体を掲載します。

① `<template>` 側

```
<template>
  <div class="admin-sub-container">
    <div class="control-panel">
      <div class="search-box">
        <label for="searchQuery" class="control-label">アカウント検索</label>
        <input
          id="searchQuery"
          type="text"
          v-model="searchQuery"
          placeholder="名前やIDで検索（半角スペース区切りで複数ワード検索対応）"
          class="input-search"
        />
      </div>
    </div>
  </div>
```

```

</div>

<div class="per-page-box">
  <label for="perPage" class="control-label">表示件数</label>
  <select id="perPage" v-model="perPage" class="select-per-page">
    <option :value="10">10件</option>
    <option :value="50">50件</option>
    <option :value="100">100件</option>
    <option :value="500">500件</option>
  </select>
</div>
</div>

<div v-if="errorMessage" class="alert alert-danger">{{ errorMessage }}</div>
<div v-if="successMessage" class="alert alert-success">{{ successMessage }}</div>

<div v-if="filteredAndSortedUsers.length > 0" class="table-responsive">
  <table class="user-table">
    <thead>
      <tr>
        <th @click="toggleSort('userName')" class="sortable-header">
          ユーザー名 {{ getSortIcon('userName') }}
        </th>
        <th @click="toggleSort('userId')" class="sortable-header">
          ユーザーID (メールアドレス) {{ getSortIcon('userId') }}
        </th>
        <th @click="toggleSort('isAuth')" class="sortable-header">
          権限 {{ getSortIcon('isAuth') }}
        </th>
        <th @click="toggleSort('quitDemand')" class="sortable-header">
          ステータス {{ getSortIcon('quitDemand') }}
        </th>
        <th>権限の変更</th>
        <th>退会申請</th>
      </tr>
    </thead>

```

```

<tbody>
  <tr v-for="user in paginatedUsers" :key="user.accountId" :class="{ 'row-quit-pending': user.quitDemand === 1 }">
    <td><strong>{{ user.userName }}</strong></td>

    <td>{{ user.userId }}</td>

    <td>
      <span :class="['badge', user.isAuth === 1 ? 'badge-admin' : 'badge-general']">
        <span v-if="user.isExective">統括管理者</span>
        <span v-if="!user.isExective && user.isAuth === 1">管理者</span>
        <span v-if="user.isAuth !== 1">一般ユーザー</span>
      </span>
    </td>

    <td>
      <span v-if="user.quitDemand === 1" class="badge badge-warning animate-pulse">
        退会申請中
      </span>
      <span v-else class="badge badge-normal">
        正常稼働
      </span>
    </td>

    <td>
      <span v-if="user.accountId === currentLoginUserId" class="role-text-frozen">
        <span v-if="user.isExective" class="self-badge">統括管理者</span>
        <span v-if="!user.isExective && user.isAuth === 1" class="self-badge">管理者</span>
        <span v-if="user.isAuth !== 1" class="self-badge">一般ユーザー</span>
        <span v-if="user.accountId === currentLoginUserId" class="self-badge">(あなた)</span>
      </span>
      <select
        v-else
        v-model="user.role"
        @change="handleRoleChange(user.accountId, user.role)"
        class="role-select-box"
      >
        <option value="ADMIN">管理者</option>
        <option value="USER">一般ユーザー</option>
      </select>
    </td>
  </tr>

```

```

        <td>
          <button
            v-if="user.quitDemand === 1 && user.userId !== 'admin@example.com'"
            @click="handleApproveQuit(user)"
            class="btn-approve"
          >
            退会を承認する
          </button>
          <span v-else-if="user.userId === 'admin@example.com'" class="text-muted-info">-</span>
          <span v-else class="text-muted">-</span>
        </td>
      </tr>
    </tbody>
  </table>
</div>

<div v-if="filteredAndSortedUsers.length > 0" class="pagination-panel">
  <div class="pagination-info">
    全 {{ filteredAndSortedUsers.length }} 件中 {{ startIndex + 1 }} ~ {{ endIndex }} 件目を表示
  </div>
  <div class="pagination-buttons">
    <button :disabled="currentPage === 1" @click="currentPage--" class="btn-page">◀ 前へ</button>
    <button
      v-for="page in totalPages"
      :key="page"
      @click="currentPage = page"
      :class="['btn-page-number', { 'active': currentPage === page }]"
    >
      {{ page }}
    </button>
    <button :disabled="currentPage === totalPages" @click="currentPage++" class="btn-page">次へ ▶</button>
  </div>
</div>

<div v-else class="empty-box">
  条件に一致するアカウントが存在しません。
</div>
</div>
</template>

```

② <script setup> 側：変数設定とAPI送信関数の追加

```
<script setup>
import { computed, onMounted, ref, watch } from 'vue';
import apiClient from '../api';

const users = ref([]);
const errorMessage = ref('');
const successMessage = ref('');

const searchQuery = ref('');
const perPage = ref(10);
const currentPage = ref(1);
const sortKey = ref('quitDemand');
const sortOrder = ref('desc');

const fetchUserList = async () => {
  try {
    const response = await apiClient.get('/users/admin/list');
    const fetchedUsersInfo = response.data.map((user, index) => {
      if (index == 0 && user.isAuth === 1) {
        return { ...user, isExective: true };
      } else {
        return { ...user, isExective: false };
      }
    });
    users.value = fetchedUsersInfo;
  } catch (error) {
    errorMessage.value = 'アカウント一覧の取得に失敗しました。';
  }
};
```

```

onMounted(() => {
  // ローカルストレージなどから、現在ログインしているユーザーの情報をロードして変数にセット
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    currentLoginUserId.value = user.accountId;
  }

  // 既存のユーザー一覧ロード処理（記述があればそのまま呼び出し）
  if (typeof fetchUserList === 'function') {
    fetchUserList();
  }
});

const filteredAndSortedUsers = computed(() => {
  let result = [...users.value];
  if (searchQuery.value.trim()) {
    const keywords = searchQuery.value.replace(/ /g, ' ').toLowerCase().split(' ').filter(w => w);
    result = result.filter(user => {
      const targetText = `${user.userName} ${user.userId}`.toLowerCase();
      return keywords.every(keyword => targetText.includes(keyword));
    });
  }
  result.sort((a, b) => {
    let modifier = sortOrder.value === 'desc' ? -1 : 1;
    let valA = a[sortKey.value];
    let valB = b[sortKey.value];
    if (typeof valA === 'string') valA = valA.toLowerCase();
    if (typeof valB === 'string') valB = valB.toLowerCase();
    if (valA < valB) return -1 * modifier;
    if (valA > valB) return 1 * modifier;
    return 0;
  });
  return result;
});

const totalPages = computed(() => Math.ceil(filteredAndSortedUsers.value.length / perPage.value) || 1);
const startIndex = computed(() => (currentPage.value - 1) * perPage.value);
const endIndex = computed(() => {
  const end = startIndex.value + perPage.value;
  return end > filteredAndSortedUsers.value.length ? filteredAndSortedUsers.value.length : end;
});

```

```

const paginatedUsers = computed(() => filteredAndSortedUsers.value.slice(startIndex.value, endIndex.value));

watch([searchQuery, perPage], () => { currentPage.value = 1; });

const toggleSort = (key) => {
  if (sortKey.value === key) {
    sortOrder.value = sortOrder.value === 'asc' ? 'desc' : 'asc';
  } else {
    sortKey.value = key;
    sortOrder.value = 'asc';
  }
};

const getSortIcon = (key) => {
  if (sortKey.value !== key) return '↕';
  return sortOrder.value === 'asc' ? '▲' : '▼';
};

const handleApproveQuit = async (targetUser) => {
  if (!confirm(`本当に「${targetUser.userName}」さんの退会申請を承認しますか？\nこの操作を行うと、アカウント情報は完全に物理削除されます。`)) return;
  try {
    await apiClient.delete(`/users/approve-quit/${targetUser.accountId}`);
    successMessage.value = `「${targetUser.userName}」さんの退会承認が完了しました。`;
    await fetchUserList();
  } catch (error) {
    errorMessage.value = '承認処理に失敗しました。';
  }
};

// 状態管理変数
const currentLoginUserId = ref(''); // ログイン中の自分のアカウントID

```

```

// ロール表記を日本語に変換する補助関数
const formatRoleLabel = (roleStr) => {
  if (roleStr === 'SUPER_ADMIN') return '統括管理者';
  if (roleStr === 'ADMIN') return '管理者';
  return '一般ユーザー';
};

// セレクトボックスが変更されたときにバックエンドへPOSTする関数
const handleRoleChange = async (targetAccountId, newRole) => {
  if (!confirm('このユーザーの所属権限を変更してもよろしいですか？\n(変更後、即座にアクセス制限が切り替わります)')) {
    // キャンセルされた場合は、一覧を再読込してセレクトボックスの表示を元に戻します
    fetchUserList();
    return;
  }

  try {
    // バックエンドの新設APIへ、自分自身のIDと対象のIDを添えて送信
    const response = await apiClient.post('/users/admin/update-role', {
      loginAccountId: currentLoginUserId.value,
      targetAccountId: targetAccountId,
      newRole: newRole
    });

    alert(response.data); // 「ユーザーの権限を正常に更新しました。」を表示
    fetchUserList();      // テーブルをリフレッシュ
  } catch (error) {
    console.error('権限更新エラー:', error);
    alert(error.response?.data || '権限の変更に失敗しました。');
    fetchUserList(); // エラー時も表示をリセット
  }
};
</script>

```

連動の確認とデバッグポイント

修正ファイルをそれぞれ配置し、Spring Bootの再起動を行ったあと、以下のステップで動作を確かめてみてください。

- **統括管理者でログインしたとき:**
 - 自分自身の行にはセレクトボックスが出現せず、代わりに「(あなた)」というバッジが点灯することを確認します（安全ガードの作動）。
 - 自分以外の従業員の行にはセレクトボックスが表示され、試しに「一般ユーザー」から「管理者」へ切り替えた際、確認アラートを経て「正常に更新しました」と表示されれば成功です！
- **一般の管理者やユーザーでログインしたとき:**
 - 他のすべてのユーザーの行も含めて、セレクトボックスが一切出現せず、すべて通常のテキスト表示（編集不可）になっていれば完璧です。

これで、統括管理者だけに許された強力なコマンドでありながら、システムを崩壊させないための安全機構がマウントされました！